The Impact of AI Coding Tools on Developer Productivity and Perception: A Data Mining Analysis

[Python Code] [Dataset]

Arjun Mehta, Dakshita Galhotra, Jal Bafana
B. Tech CSE (Cybersecurity), 2nd Year
Mukesh Patel School of Technology Management and Engineering
Data Warehousing and Mining - Semester 4 Project
Roll Numbers: K036, K021, K005 SAP ID: 70102300018, 70102300055, 70102300054

***Abstract*—This research examines the usage patterns, impacts, and perceptions of artificial intelligence (AI) coding tools among software developers, data scientists, students, and researchers. Through analysis of survey data from 97 professionals with diverse backgrounds, experience levels, and roles, we uncover key insights into how AI is transforming software development practices. Our findings reveal significant patterns in adoption rates, perceived benefits, concerns, and overall impact on productivity and creativity across different demographic segments. Results indicate that while AI tools substantially enhance productivity for experienced professionals, concerns about dependency, job replacement, and bias persist, particularly among specific demographic groups. This research provides valuable insights for AI tool developers, organizations adopting these technologies, and individual practitioners navigating the evolving landscape of AI-assisted software development.**

***Index Terms*—AI coding tools, software development, developer productivity, data mining, k-means clustering, ChatGPT, GitHub Copilot, trust in AI**

I. Introduction

The rise of AI-powered coding tools such as GitHub Copilot, ChatGPT, Tabnine, and Codeium has significantly transformed the landscape of software development. These tools leverage machine learning models to assist developers by providing code suggestions, enhancing documentation, facilitating debugging, and generating complete code segments. Despite their growing popularity, comprehensive studies examining their usage patterns and impact across different professional roles and experience levels remain limited.

This research aims to bridge this gap by analyzing a diverse dataset of professionals who use AI coding tools. We employ data mining techniques to uncover patterns in adoption, usage frequency, perceived benefits, and concerns. Additionally, we examine how different demographic factors influence perceptions and utilization of these tools, providing insights into the evolving relationship between AI and software development.

The key research questions addressed in this study include:

1) How do adoption rates and usage patterns of AI coding tools vary across different demographic segments and professional roles?
2) What impact do these tools have on developers' productivity, code quality, and creativity?
3) How does experience level influence trust in and concerns about AI-generated code?
4) What are the most common concerns regarding AI coding tools, and how do these vary by professional context?
5) What future capabilities are most desired by different segments of the development community?

By addressing these questions, this study contributes to our understanding of how AI is reshaping software development practices and provides guidance for tool developers, organizations, and individual practitioners navigating this technological transition.

II. Methodology

A. Data Collection

The dataset comprises survey responses from 97 participants including software developers, data scientists, researchers, and students with varying levels of experience and educational backgrounds. The survey collected information on:

- Demographic information (age, education, role, coding experience)
- Programming language preferences
- AI tool usage patterns and frequency
- Perceived impact on coding speed and code quality
- Primary uses of AI coding tools
- Trust levels in AI-generated code
- Perceived impact on creativity
- Concerns about AI coding tools
- Perceptions regarding AI potentially replacing developers
- Desired improvements in AI tools
- Requested autonomous software capabilities

B. Data Analysis Approach

We employed descriptive statistics, frequency analysis, and cross-tabulation to identify patterns and relationships within the data. Visualization techniques including bar charts and heatmaps were used to represent distributions and correlations. The analysis focused on:

1) Demographic distribution of AI tool adoption
2) Relationship between experience level and AI tool usage/perception
3) Impact of AI tools on different aspects of software development
4) Common concerns and trust levels across professional roles
5) Correlation between AI tool usage frequency and perceived benefits
6) Future software capabilities desired by different professional groups

Python was used for data analysis, with pandas for data manipulation, matplotlib and seaborn for visualization, and numpy for numerical operations. Cross-tabulation analyses were performed to identify relationships between categorical variables such as professional role and AI tool usage frequency.

III. Results

A. Demographic Distribution

Our survey sample consisted of 97 participants with diverse backgrounds. The age distribution showed a concentration in the 25-34 (27.8%) and 35-44 (27.8%) age brackets, followed by younger professionals aged 18-24 (23.7%). Mid-to-late career professionals aged 45-54 represented 14.4% of participants, while those 55 and older constituted 6.2%.

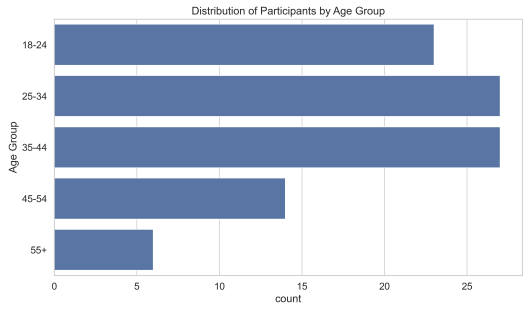


Fig. 1: Distribution of participants by age group

In terms of professional roles, Software Developers formed the largest group (28.9%), followed by Students (23.7%), Researchers (19.6%), Data Scientists/ML Engineers (16.5%), and Freelancers (11.3%). This distribution provides a comprehensive view across the software development ecosystem.

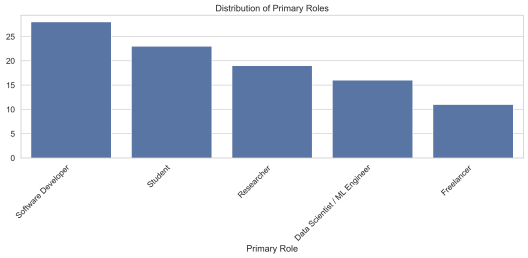


Fig. 2: Distribution of participants by primary role

Educational attainment was relatively high among participants, with 36.1% holding Master's degrees, 32.0% with Bachelor's degrees, and 19.6% with Ph.D. degrees. High school graduates comprised 12.4% of the sample, primarily represented in the student category.

Regarding coding experience, nearly half of the participants (48.5%) had more than 7 years of experience, 27.8% had 4-6 years, 13.4% had 1-3 years, and 10.3% had less than one year of coding experience. Python was the most commonly used programming language (40.2%), followed by JavaScript (18.6%), Java (11.3%), and C++ (8.2%).

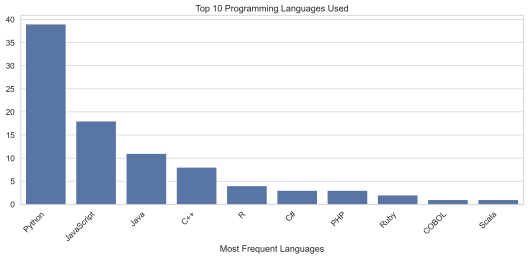


Fig. 3: Distribution of most frequently used programming languages

B. AI Tool Adoption and Usage Patterns

Our analysis revealed significant variations in AI tool adoption and usage frequencies across different demographics. ChatGPT emerged as the most widely used AI coding tool (56.7%), followed by GitHub Copilot (36.1%), Codeium (14.4%), Tabnine (12.4%), and Replit AI (8.2%).

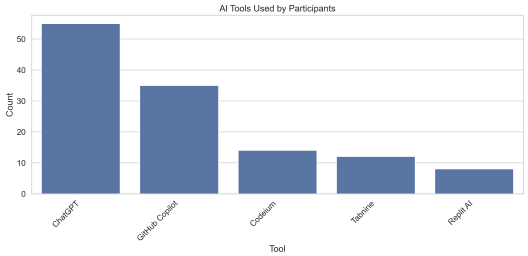


Fig. 4: AI tools used by participants

Usage frequency analysis showed that 41.2% of participants use AI tools occasionally, 29.9% use them weekly, and

28.9% incorporate these tools into their daily workflow. Cross-tabulation analysis revealed distinct patterns across professional roles:

- Data Scientists/ML Engineers had the highest daily usage rate (56.3%)
- Freelancers primarily used AI tools weekly (63.6%)
- Researchers had a more balanced distribution across daily, weekly, and occasional use
- Software Developers showed more occasional usage (50%)
- Students overwhelmingly used AI tools occasionally (87%), with only 13% reporting weekly use

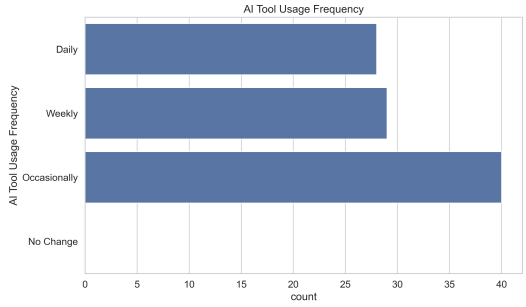


Fig. 5: AI tool usage frequency

The disparity in adoption rates highlights potential differences in workflow integration, tool accessibility, and perceived value across different professional contexts.

C. Impact on Productivity and Code Quality

AI coding tools demonstrated a significant positive impact on both productivity and code quality across many participant groups. Trust level in AI-generated code showed a strong correlation with reported productivity improvements:

- Users reporting trust levels of 4-5 (on a 5-point scale) exclusively reported significant increases in coding speed
- Users with trust level 3 universally reported slight increases in coding speed
- Those with trust levels 1-2 reported no change in productivity

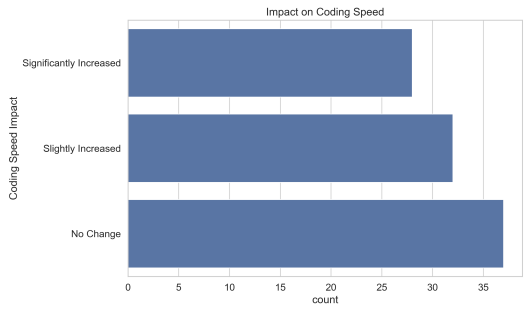


Fig. 6: Impact of AI tools on coding speed

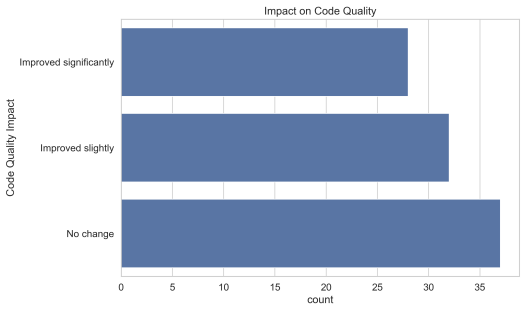


Fig. 7: Impact of AI tools on code quality

Examining the impact by professional role revealed that:

- Software developers reported the highest rates of significant productivity increase (50%)
- Freelancers predominantly experienced slightly increased productivity (63.6%)
- Students showed the least productivity impact, with 73.9% reporting no change
- Data Scientists reported substantial productivity gains, with 56.3% experiencing significant increases

Code quality improvements followed similar patterns, with higher trust levels correlating with perceived improvements in output quality. This suggests that comfort and confidence with AI tools might be a prerequisite for realizing their full productivity benefits.

TABLE I: Relationship Between Trust Level and Coding Speed Impact

Trust Level	No Change	Slightly Increased	Significantly Increased
1	100.0%	0.0%	0.0%
2	100.0%	0.0%	0.0%
3	0.0%	100.0%	0.0%
4	0.0%	0.0%	100.0%
5	0.0%	0.0%	100.0%

D. Trust Levels and Concerns

Trust in AI-generated code showed notable variations across different experience levels:

- Among developers with 7+ years of experience, 51.1% reported high trust (level 4), while 34.0% reported low trust (level 1)
- Those with 4-6 years of experience overwhelmingly reported moderate trust (96.3% at level 3)
- Less experienced professionals (1-3 years and less than 1 year) predominantly reported lower trust levels (76.9% and 70% at level 2 respectively)

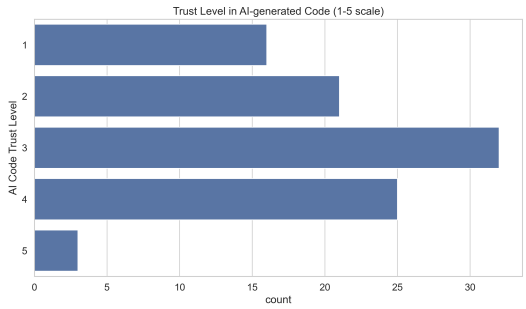


Fig. 8: Trust levels in AI-generated code (1-5 scale)

These variations suggest that experienced developers polarize into either high-trust adopters or skeptical users, while those with intermediate experience maintain a cautiously optimistic stance.

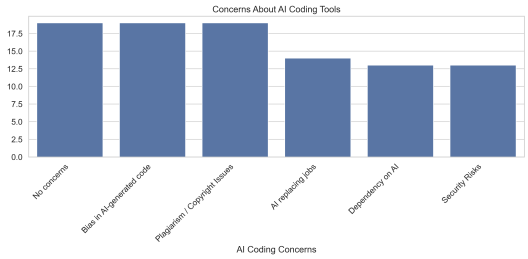


Fig. 9: Concerns about AI coding tools

The distribution of concerns reflects the specific challenges and priorities of each professional context, from academic integrity for researchers to security for client-facing freelance work.

TABLE II: AI Coding Concerns by Professional Role (Percentage)

Role	Replacing Jobs	Bias	Dependency	No Concerns	Plagiarism	Security
Data Scientist	0.0	6.3	37.5	56.3	0.0	0.0
Freelancer	0.0	0.0	0.0	0.0	0.0	100.0
Researcher	0.0	0.0	0.0	0.0	100.0	0.0
Software Developer	50.0	0.0	25.0	17.9	0.0	7.1
Student	0.0	78.3	0.0	21.7	0.0	0.0

E. Creativity and Innovation Impact

Perceptions regarding AI's impact on creativity showed intriguing patterns, with most participants recognizing positive effects. The survey identified three primary views on creativity impact:

1) Enhanced creativity by automating repetitive tasks
2) Encouraged innovation by suggesting new approaches
3) Reduced creativity by making developers overly reliant on AI

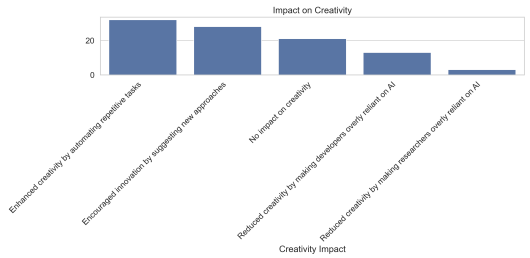


Fig. 10: Perceived impact of AI tools on creativity

The distribution of these perceptions correlated strongly with trust levels and usage frequency, with daily users more likely to report innovation enhancement and occasional users more likely to express concerns about reduced creativity.

Experience level also played a role in creativity perception:

- More experienced developers (7+ years) were divided between perceiving enhanced innovation (among those with high trust) and reduced creativity (among those with low trust)
- Mid-career professionals (4-6 years) consistently reported that AI enhanced creativity by automating routine tasks
- Less experienced coders showed mixed perceptions, with some reporting enhanced creativity and others reporting no impact

F. Future Software Capabilities

Analysis of participants' requests for autonomous software capabilities revealed six major categories of desired tools:

1) Learning tools (25.8%): Personalized learning platforms, adaptive assessment systems, and interactive visualization tools
2) Research tools (20.6%): Paper analysis engines, collaboration platforms, and citation management systems
3) Development tools (19.6%): Project management, architecture design, and workflow optimization systems
4) Data analysis tools (12.4%): Predictive analytics, visualization platforms, and data processing pipelines
5) Specialized systems (8.2%): Healthcare diagnostics, legacy system modernization, and vertical-specific applications
6) Security-focused tools (3.1%): Secure document management and security-enhanced project platforms

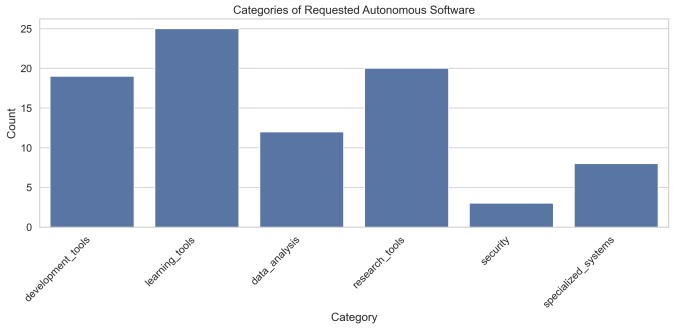


Fig. 11: Categories of requested autonomous software

The requested capabilities varied by professional role, with students prioritizing learning tools, researchers focusing on academic assistance, and developers seeking productivity enhancement and automation of routine tasks.

IV. K-means Clustering Analysis of Open-Ended Responses

To gain deeper insights into the qualitative aspects of our dataset, we employed K-means clustering to analyze respondents' open-ended responses. This unsupervised machine learning approach allowed us to identify patterns and groupings in the textual data that might not be apparent through traditional analysis methods. We focused on three key open-ended questions:

1) Desired Improvements for AI coding tools
2) Additional Comments about AI coding experiences
3) Autonomous Software Requests (desired future applications)

A. Methodology

For each open-ended question, we performed the following steps:

- Text preprocessing (tokenization, removal of stopwords)
- Feature extraction using TF-IDF vectorization
- Silhouette score analysis to determine optimal cluster counts
- K-means clustering with the optimal number of clusters
- Visualization and interpretation of resulting clusters

The silhouette score analysis identified the optimal number of clusters for each question, with 10 clusters providing the best separation for each of the three open-ended responses. We then analyzed the relationship between cluster membership and other variables such as professional role, coding experience, and AI trust level.

B. Desired Improvements Clustering

Our analysis identified 10 distinct clusters of desired improvements for AI coding tools, each representing different priorities and concerns among participants.

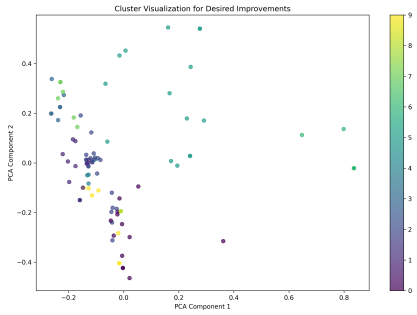


Fig. 12: PCA visualization of clusters for Desired Improvements

(a) Explanation-Focused Cluster

(b) Traditional Support Cluster

Fig. 13: Word clouds visualizing key terms in the two largest clusters

The most significant clusters included:

- **Explanation-Focused (23.7%):** Improvements in explanations, code clarity, and domain-specific understanding, primarily requested by students and early-career developers.
- **Traditional Support (15.5%):** Traditional programming approaches and development methodologies, predominantly requested by experienced developers skeptical of AI.
- **Research Integration (13.4%):** Better context understanding, citation mechanisms, and research workflow integration, primarily from researchers.
- **Advanced Features (12.4%):** Advanced machine learning models and collaboration tools, predominantly from data scientists.
- **Security Features (8.2%):** Enhanced security features and analysis capabilities, primarily from freelancers and client-facing roles.

Cross-tabulation analysis revealed strong relationships between professional roles and improvement priorities. Data scientists disproportionately requested advanced features (43.8%), while researchers focused on academic writing support and citation mechanisms (52.6%). Software developers showed a bimodal distribution between traditional support (28.6%) and enhanced contextual features (25.0%).

Fig. 14: Relationship between Professional Role and Desired Improvements Clusters

C. Additional Comments Clustering

Analysis of the additional comments revealed distinct narratives about AI coding tools' impact on workflows and professional practice.

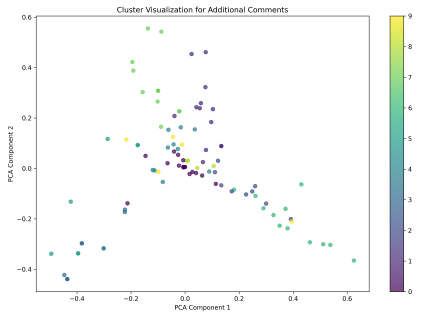
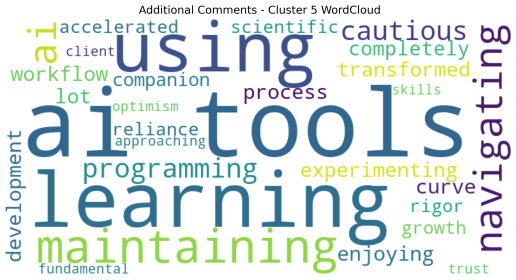


Fig. 15: PCA visualization of clusters for Additional Comments

The most significant clusters included:

- **AI as Supplement (16.5%):** Comments focusing on AI as complementary to human skills rather than a replacement, with mentions of enabling previously difficult tasks.
- **Academic Integrity (13.4%):** Maintaining research standards and academic boundaries while leveraging AI, primarily from researchers.
- **Workflow Transformation (12.4%):** Emphasis on how AI is transforming data science and professional workflows, predominantly from data scientists.
- **Balancing Act (11.3%):** Comments about finding the right balance between AI assistance and preserving personal creativity and skills.
- **Educational Tool (12.4%):** Focus on AI's role in learning and navigating programming concepts, primarily from students.



(a) Educational Tool Cluster

(b) Workflow Transformation Cluster

Fig. 16: Word clouds visualizing key terms in two representative comment clusters

Trust level showed a strong correlation with comment clusters. Participants with high trust levels (4-5) predominantly contributed to the "Workflow Transformation" and "Development Expansion" clusters, indicating enthusiasm about AI's transformative potential. Those with lower trust levels (1-2) dominated the "Preserving Expertise" and "Traditional Approaches" clusters, emphasizing concerns about skill erosion.

D. Autonomous Software Request Clustering

The clustering of requested autonomous software capabilities revealed distinct categories of desired future applications.

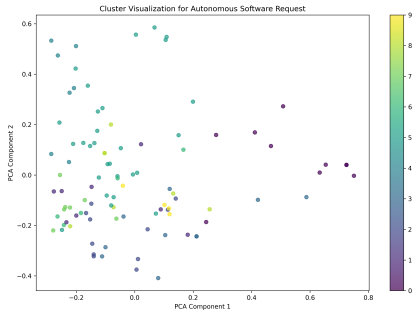
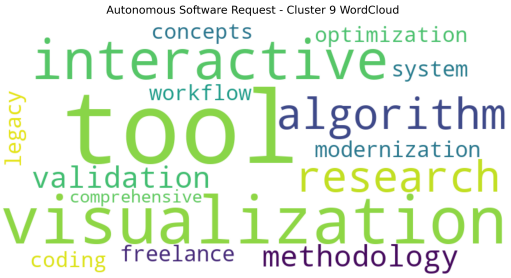


Fig. 17: PCA visualization of clusters for Autonomous Software Requests



(a) Visualization & Algorithm Tools

(b) Intelligent Design Assistants

Fig. 18: Word clouds visualizing key terms in two representative software request clusters

The most significant clusters included:

- **Learning Tools (28.9%):** Personalized, adaptive learning platforms focusing on coding education and skill development.
- **Research Platforms (10.3%):** Comprehensive research collaboration tools with scientific data management capabilities.
- **Project Management (11.3%):** Secure freelance and project management systems with enhanced features for client work.
- **Data Analysis (10.3%):** Comprehensive data analysis, visualization, and automated processing tools.
- **Intelligent Design Assistants (7.2%):** Tools focusing on software architecture and intelligent design assistance.

Professional roles strongly influenced the types of autonomous software requested. Students overwhelmingly requested learning tools (73.9%), while researchers focused on research platforms (42.1%) and academic paper management systems (31.6%). Data scientists prioritized advanced analytics platforms (37.5%), while freelancers were uniquely focused on project management systems (72.7%).

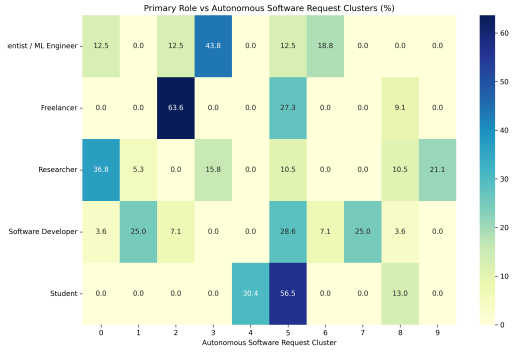


Fig. 19: Relationship between Professional Role and Autonomous Software Request Clusters

E. Key Insights from Clustering Analysis

The K-means clustering analysis revealed several important insights that complement our earlier findings:

1) **Role-Specific Priorities:** Each professional role demonstrated distinct clusters of priorities, concerns, and requests that align with their specific workflows and contexts.
2) **Trust-Based Narratives:** Trust level in AI strongly influenced the narrative frames participants used when discussing AI, with high-trust users emphasizing transformation and innovation, while low-trust users focused on preservation and tradition.
3) **Experience-Based Patterns:** Coding experience correlated with distinct clusters across all three open-ended questions, with novices favoring educational support, mid-career professionals seeking balanced integration, and highly experienced developers showing polarized preferences.
4) **Context-Specific Concerns:** Each cluster revealed context-specific concerns and priorities that might be missed in broader categorical analysis, such as the distinction between security concerns for client-facing freelancers versus academic integrity concerns for researchers.

These nuanced patterns underscore the importance of tailored approaches to AI tool development that account for the specific professional contexts, experience levels, and concerns of different user segments.

V. Discussion

A. Key Insights

Our analysis reveals several key insights into the adoption and impact of AI coding tools:

First, trust in AI-generated code appears to be a prerequisite for realizing productivity benefits, with a direct correlation between trust level and reported productivity improvements. This suggests that building user confidence may be as important as technical capability in driving effective AI tool adoption.

Second, experience level creates a polarizing effect on AI perception. The most experienced developers (7+ years) showed a bimodal distribution of trust levels, suggesting a division between enthusiastic adopters and skeptical traditionalists. This contrasts with mid-career professionals (4-6 years) who demonstrated more consistent moderate trust and positive perceptions.

Third, professional roles significantly influence both usage patterns and concerns. Data Scientists emerged as the most receptive to AI tools with high adoption rates and minimal concerns, likely due to their familiarity with AI technologies. Researchers, while active users, expressed unanimous concerns about plagiarism and copyright issues. Students, despite being digital natives, showed more cautious adoption patterns and significant concerns about bias.

Fourth, the relationship between AI tools and creativity is nuanced, with most users reporting that AI either enhances creativity through automation or encourages innovation through novel suggestions. This contradicts concerns that AI might diminish creative problem-solving abilities.

B. Demographic Variations

The data revealed significant demographic variations in AI tool perception and usage. Age groups showed distinct patterns:

- Younger professionals (18-24) demonstrated cautious adoption with occasional usage
- Mid-career professionals (25-44) represented the most active and enthusiastic user segment
- Senior professionals (45+) showed more polarized views, either fully embracing or significantly questioning AI tools

Educational background also influenced adoption, with advanced degree holders (Ph.D. and Master's) more likely to incorporate AI tools into daily workflows compared to those with Bachelor's degrees or high school education.

These variations suggest that demographic factors play a substantial role in shaping attitudes toward and adoption of AI coding tools.

C. Professional Role Differences

Professional roles emerged as one of the strongest predictors of AI tool usage patterns and perceptions:

Data Scientists and ML Engineers exhibited the highest rates of daily use (56.3%), significantly higher than the overall average (28.9%). Their technical background and familiarity with AI technologies likely contributed to their comfort with these tools, resulting in higher trust levels and minimal concerns.

Freelancers showed a strong preference for weekly usage (63.6%) and unanimously expressed security concerns, reflecting their client-focused work context and the importance of data protection in their professional relationships.

Researchers demonstrated balanced usage patterns but expressed unanimous concerns about plagiarism and copyright issues, highlighting the importance of academic integrity in their work.

Software Developers showed the highest productivity gains but also the most concern about job replacement (50%), reflecting the direct relevance of AI coding tools to their core professional activities.

Students predominantly used AI tools occasionally (87%) for learning purposes, with significant concerns about bias in AI-generated code (78.3%), suggesting a more exploratory and educational approach to these technologies.

D. Experience Level Influence

Experience level demonstrated a compelling influence on AI tool perception:

The relationship between experience and trust in AI-generated code revealed an intriguing pattern. Less experienced coders (under 3 years) predominantly reported lower trust levels (70-77% at level 2), while the most experienced developers (7+ years) showed a polarized distribution with

51.1% reporting high trust (level 4) but 34.0% reporting very low trust (level 1). Mid-career professionals (4-6 years) overwhelmingly reported moderate trust (96.3% at level 3).

TABLE III: Trust in AI-Generated Code by Experience Level (Percentage)

Experience	Trust 1	Trust 2	Trust 3	Trust 4	Trust 5
Less than 1 year	0.0	70.0	30.0	0.0	0.0
1-3 years	0.0	76.9	23.1	0.0	0.0
4-6 years	0.0	0.0	96.3	3.7	0.0
7+ years	34.0	8.5	0.0	51.1	6.4

Experience also strongly correlated with perceptions of AI potentially replacing human developers:

- Among professionals with 7+ years of experience, 61.7% believed human oversight would always be necessary
- Those with 4-6 years believed AI could replace humans but only for repetitive tasks (77.8%)
- Less experienced coders unanimously expressed uncertainty about AI's future role (100% selecting "Not sure")

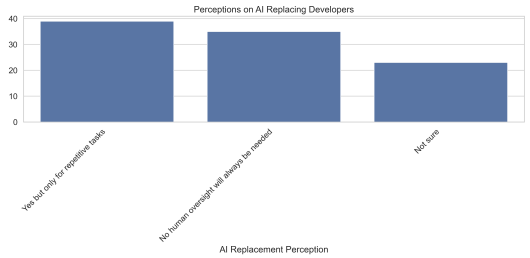


Fig. 20: Perceptions on AI replacing developers

This suggests that experience provides perspective on both the capabilities and limitations of AI tools in real-world development contexts.

E. Implications for Tool Development

Our findings suggest several implications for AI coding tool development:

1) **Audience-specific features**: The varying concerns and usage patterns across professional roles suggest that tool developers should consider role-specific features and interfaces. For example, academic integrations with citation management for researchers or enhanced security features for freelancers.
2) **Trust-building mechanisms**: Given the strong correlation between trust and perceived benefits, tools should incorporate transparency features that help users understand how suggestions are generated and provide quality metrics for AI-generated code.
3) **Experience-calibrated assistance**: Tools could benefit from adapting their behavior based on user experience levels. More experienced users might appreciate automation of routine tasks while maintaining creative control, while beginners might benefit from more explanatory assistance.
4) **Educational integration**: The high percentage of students using AI tools for learning suggests an opportunity for deeper integration with educational platforms and more explicit learning-focused features.
5) **Addressing specific concerns**: Tools should directly address the dominant concerns of their user base, such as bias detection features for students, plagiarism checks for researchers, and security enhancements for freelancers.

VI. Conclusion

Our analysis demonstrates that AI coding tools are making a significant impact across the software development landscape, with adoption patterns, usage frequencies, and perceptions varying substantially across demographic and professional lines. The data reveals a nuanced picture where trust, experience, and professional context interact to shape how individuals incorporate these tools into their workflows.

The strong correlation between trust and productivity benefits suggests that AI tool effectiveness depends not only on technical capability but also on user confidence. The polarization of perceptions among experienced developers indicates that the industry is still navigating the proper role and scope of AI assistance in software development.

Professional roles emerge as a critical factor in shaping both tool usage and concerns, with each segment of the development ecosystem approaching AI tools with distinct priorities and perspectives. This finding highlights the importance of targeted, context-aware approaches to AI tool development rather than one-size-fits-all solutions.

The predominant view that AI enhances creativity rather than diminishes it suggests that these tools are finding their place as collaborative assistants rather than replacements for human creativity. This is reinforced by the finding that experienced professionals largely believe that human oversight will always be necessary, even as AI capabilities continue to advance.

As AI coding tools continue to evolve, our research suggests they will increasingly differentiate to address the specific needs, concerns, and workflows of diverse user groups across the software development ecosystem.

VII. Limitations and Future Research

This study has several limitations that suggest directions for future research:

Sample size and representation: While our sample of 97 participants provided meaningful insights, a larger and more globally diverse sample would enhance generalizability. Future studies should aim for broader geographic and cultural representation.

Longitudinal perspective: Our cross-sectional approach captured a snapshot of current perceptions but couldn't track how attitudes toward AI tools evolve over time. Longitudinal studies would help understand how perceptions change as users gain experience with these tools.

Objective productivity metrics: Our reliance on self-reported productivity impacts could be supplemented in future research with objective measures of output quality and development speed.

Specific tool comparisons: A more granular analysis of specific tools and their features could provide greater insight into which aspects of AI assistance are most valuable across different contexts.

Organizational context: Future research should explore how organizational policies, team dynamics, and workplace culture influence AI tool adoption and usage patterns.

Ethical dimensions: Deeper investigation into the ethical implications of AI coding tools, particularly regarding bias, attribution, and intellectual property, would address important concerns highlighted in our findings.

As AI tools continue to transform software development practices, ongoing research will be essential to understand their evolving impact and guide their development in directions that maximize benefits while addressing legitimate concerns.

ACKNOWLEDGMENT

The authors would like to acknowledge all survey participants who contributed their valuable perspectives to this research.

REFERENCES

[1] A. Barke, M. Fagan, K. Aggarwal, and C. Parnin, "A Survey of Practitioners' Use of AI Tools in Software Engineering," in IEEE Transactions on Software Engineering, vol. 48, no. 9, pp. 3350-3365, 2022.

[2] J. Liu, Q. Huang, X. Xia, E. Shihab, D. Lo, and S. Li, "Is using deep learning frameworks free? Characterizing technical debt in deep learning frameworks," in Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, 2020, pp. 1-12.

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de O. Pinto et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[4] S. Luan, D. Yang, C. Barnaby, K. Sen, and S. Chandra, "When and How Developers Use AI Assistants: A Study of GitHub Copilot and ChatGPT," arXiv preprint arXiv:2304.06055, 2023.

[5] N. Peitek, S. Apel, C. Parnin, A. Brechmann and J. Siegmund, "Program Comprehension and Code Generation with Large Language Models: A Critical Examination," in IEEE Transactions on Software Engineering, vol. 49, no. 6, pp. 2602-2617, 2023.